Virtual Memory

Virtual Memory - Address Translation;

Page Table; Translation Lookaside Buffer; Memory Protection

Virtual Memory

- Most modern computer systems use a *hard drive* made of magnetic or solid-state storage as the lowest level in the memory hierarchy (see Figure 8.4).
- Compared with the ideal large, fast, cheap memory, a hard drive is large and cheap but terribly slow.
- It provides a much larger capacity than is possible with a cost-effective main memory (DRAM).
- However, if a significant fraction of memory accesses involve the hard drive, performance is poor.

- Figure 8.19 shows a hard drive made of magnetic storage, also called a *hard disk*, with the lid of its case removed.
- As the name implies, the hard disk contains one or more rigid disks or *platters*, each of which has a *read/write head* on the end of a long triangular arm.
- The head moves to the correct location on the disk and reads or writes data magnetically as the disk rotates beneath it.
- The head takes several milliseconds to *seek* the correct location on the disk, which is fast from a human perspective but millions of times slower than the processor.
- Hard disk drives are increasingly being replaced by solid-state drives because reading is orders of magnitude faster and they are not as susceptible to mechanical failures.

Figure 8.19 Hard disk

- The objective of adding a hard drive to the memory hierarchy is to inexpensively give the illusion of a very large memory while still providing the speed of faster memory for most accesses.
- A computer with only 16 GiB of DRAM, for example, could effectively provide 128 GiB of memory using the hard drive.
- This larger 128 GiB memory is called *virtual memory*, and the smaller 16 GiB main memory is called *physical memory*.
- We will use the term *physical memory* to refer to main memory
- Programs can access data anywhere in virtual memory, so they must use ***virtual addresses*** that specify the location in virtual memory.
- The physical memory holds a subset of most recently accessed virtual memory.
- In this way, physical memory acts as a cache for virtual memory.
- Thus, most accesses hit in physical memory at the speed of DRAM, yet the program enjoys the capacity of the larger virtual memory.

- Virtual memory systems use different terminologies for the same caching principles.
- Table 8.3 summarizes the analogous terms.
- Virtual memory is divided into ***virtual pages***, typically 4 KiB in size.
- Physical memory is likewise divided into ***physical pages*** of the same size.
- A virtual page may be located in physical memory (DRAM) or on the hard drive.
- For example, Figure 8.20 shows a virtual memory that is larger than physical memory.
- The rectangles indicate pages.
- Some virtual pages are present in physical memory, and some are located on the hard drive.
- The process of determining the physical address from the virtual address is called ***address translation***.
- If the processor attempts to access a virtual address that is not in physical memory, a ***page fault*** occurs and the operating system (OS) loads the page from the hard drive into physical memory.

Table 8.3 Analogous cache and virtual memory terms

Cache	Virtual Memory
Block	Page
Block size	Page size
Block offset	Page offset
Miss	Page fault
Tag	Virtual page number

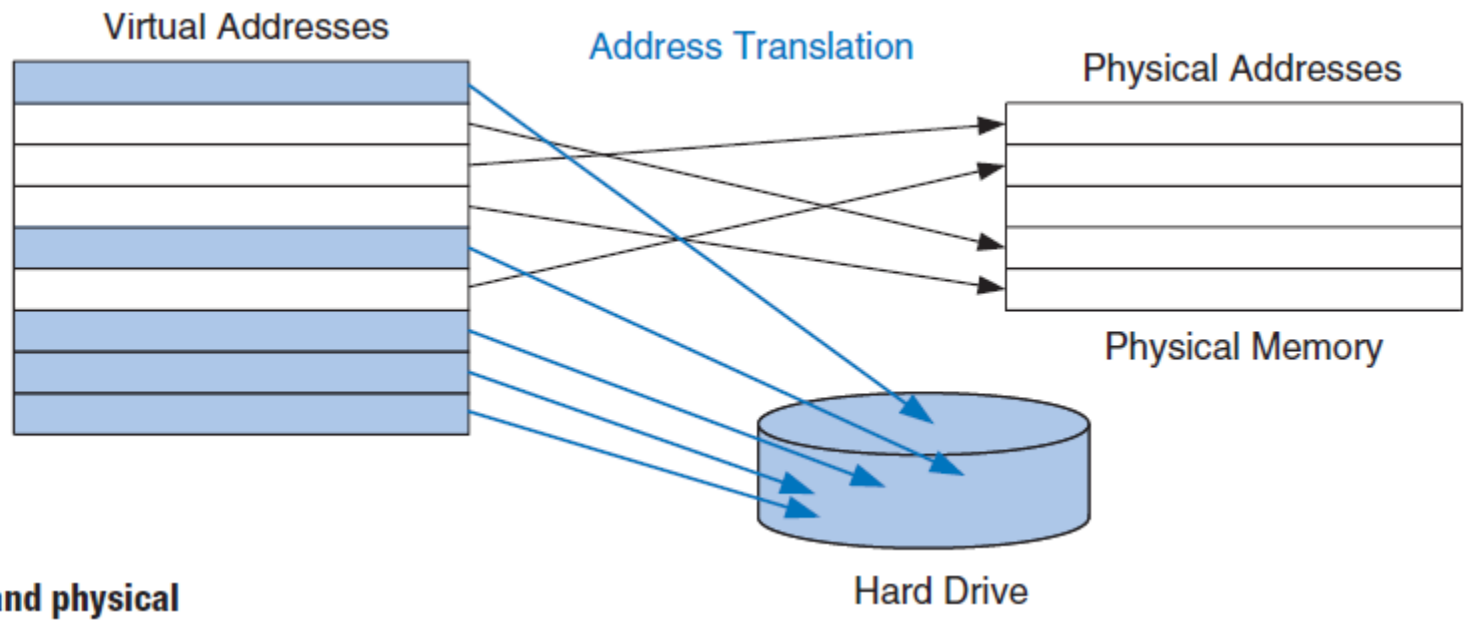

Figure 8.20 Virtual and physical pages

- To avoid page faults caused by conflicts, any virtual page can map to any physical page.
- In other words, physical memory behaves as a fully associative cache for virtual memory.
- In a conventional fully associative cache, every cache block has comparator that checks the most significant address bits against a tag to determine whether the request hits in the block.
- In an analogous virtual memory system, each physical page would need a comparator to check the most significant virtual address bits against a tag to determine whether the virtual page maps to that physical page.

- A realistic virtual memory system has so many physical pages that providing a comparator for each page would be excessively expensive.
- Instead, the virtual memory system uses a **page table** to perform address translation.
- A page table contains an entry for each virtual page, indicating its location in physical memory or that it is on the hard drive.
- Each load or store instruction requires a page table access followed by a physical memory access.
- The page table access translates the virtual address used by the program to a physical address.
- The physical address is then used to actually read or write the data.
- The page table is usually so large that it is located in physical memory.
- Hence, each load or store involves two physical memory accesses: a page table access and a data access.
- To speed up address translation, a **translation lookaside buffer** (TLB) caches the most commonly used page table entries.

Address Translation

- In a system with virtual memory, programs use virtual addresses so that they can access a large memory.
- The computer must translate these virtual addresses to either find the address in physical memory or take a page fault and fetch the data from the hard drive.
- Virtual memory and physical memory are divided into pages.
- The most significant bits of the virtual or physical address specify the virtual or physical *page number*.
- The least significant bits specify the word within the page and are called the *page offset*.

- Figure 8.21 illustrates the page organization of a virtual memory system with 2 GiB of virtual memory and 128 MiB of physical memory divided into 4 KiB pages.
- MIPS accommodates 32-bit addresses.
- With a 2 GiB = 2^{31}-byte virtual memory, only the least significant 31 virtual address bits are used; the 32nd bit is always 0.
- Similarly, with a 128 MiB = 2^{27}-byte physical memory, only the least significant 27 physical address bits are used; the upper 5 bits are always 0.

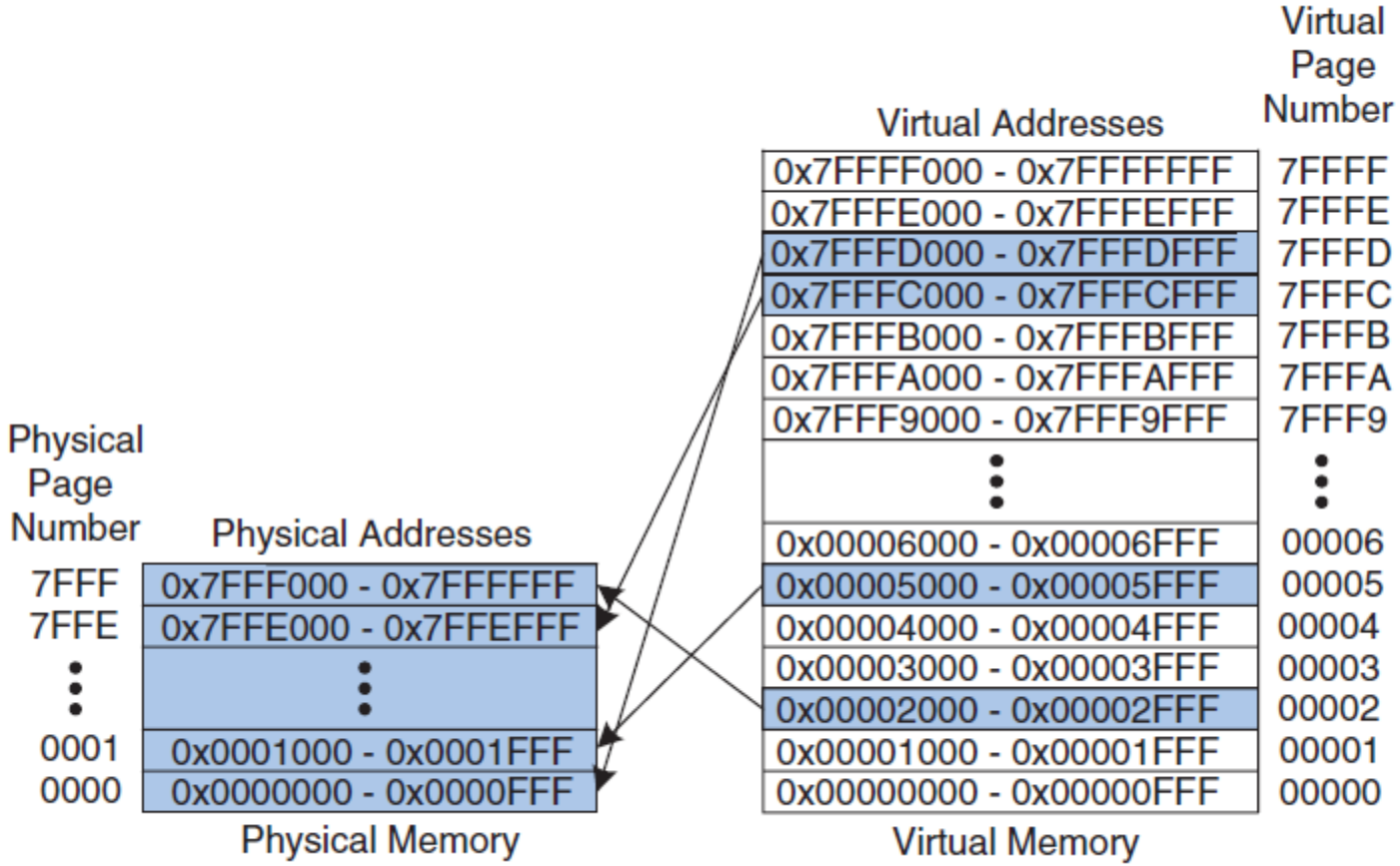

Figure 8.21 **Physical and virtual pages**

- Because the page size is 4 KiB = 2^{12} bytes, there are $2^{31}/2^{12} = 2^{19}$ virtual pages and $2^{27}/2^{12} = 2^{15}$ physical pages.
- Thus, the virtual and physical page numbers are 19 and 15 bits respectively.
- Physical memory can only hold up to 1/16th of the virtual pages at any given time.
- The rest of the virtual pages are kept on the hard drive.
- Figure 8.21 shows virtual page 5 mapping to physical page 1, virtual page 0x7FFFC mapping to physical page 0x7FFE, and so forth.
- For example, virtual address 0x53F8 (an offset of 0x3F8 within virtual page 5) maps to physical address 0x13F8 (an offset of 0x3F8 within physical page 1).
- The least significant 12 bits of the virtual and physical addresses are the same (0x3F8) and specify the page offset within the virtual and physical pages.
- Only the page number needs to be translated to obtain the physical address from the virtual address.

- Figure 8.22 illustrates the translation of a virtual address to a physical address.
- The least significant 12 bits indicate the page offset and require no translation.
- The upper 19 bits of the virtual address specify the *virtual page number* (*VPN*) and are translated to a 15-bit *physical page number* (*PPN*).

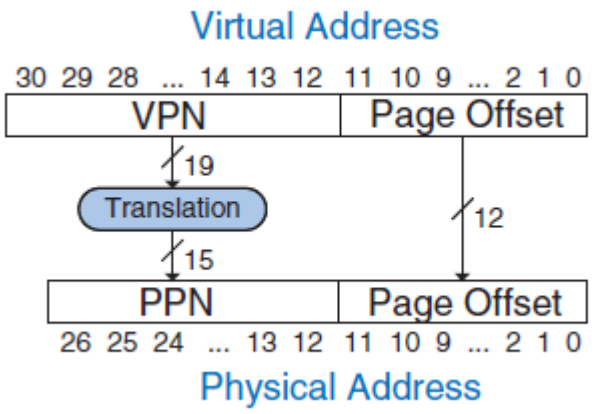

Figure 8.22 Translation from virtual address to physical address

Example 8.13 VIRTUAL ADDRESS TO PHYSICAL ADDRESS TRANSLATION

Find the physical address of virtual address 0x247C using the virtual memory system shown in Figure 8.21.

Solution

- The 12-bit page offset (0x47C) requires no translation.
- The remaining 19 bits of the virtual address give the virtual page number, so virtual address 0x247C is found in virtual page 0x2.
- In Figure 8.21, virtual page 0x2 maps to physical page 0x7FFF. Thus, virtual address 0x247C maps to physical address 0x7FFF47C.

Page Table

- The processor uses a *page table* to translate virtual addresses to physical addresses.
- The page table contains an entry for each virtual page.
- This entry contains a physical page number and a valid bit.
- If the valid bit is 1, the virtual page maps to the physical page specified in the entry.
- Otherwise, the virtual page is found on the hard drive.
- Because the page table is so large, it is stored in physical memory.
- Let us assume for now that it is stored as a contiguous array, as shown in Figure 8.23.

V	Physical Page Number	Virtual Page Number
0		7FFFF
0		7FFFE
1	0x0000	7FFFD
1	0x7FFE	7FFFC
0		7FFFB
0		7FFFA
	⋮	⋮
0		00007
0		00006
1	0x0001	00005
0		00004
0		00003
1	0x7FFF	00002
0		00001
0		00000

Page Table

Figure 8.23 The page table for Figure 8.21

- This page table contains the mapping of the memory system of Figure 8.21.
- The page table is indexed with the virtual page number (VPN).
- For example, entry 5 specifies that virtual page 5 maps to physical page 1.
- Entry 6 is invalid ($V = 0$), so virtual page 6 is located on the hard drive.

Example 8.14 USING THE PAGE TABLE TO PERFORM ADDRESS TRANSLATION

Q) Find the physical address of virtual address 0x247C using the page table shown in Figure 8.23.

Solution

- Figure 8.24 shows the virtual address to physical address translation for virtual address 0x247C.
- The 12-bit page offset requires no translation.
- The remaining 19 bits of the virtual address are the virtual page number, 0x2, and give the index into the page table.
- The page table maps virtual page 0x2 to physical page 0x7FFF.
- So, virtual address 0x247C maps to physical address 0x7FFF47C.
- The Page Table least significant 12 bits are the same in both the physical and the virtual address.

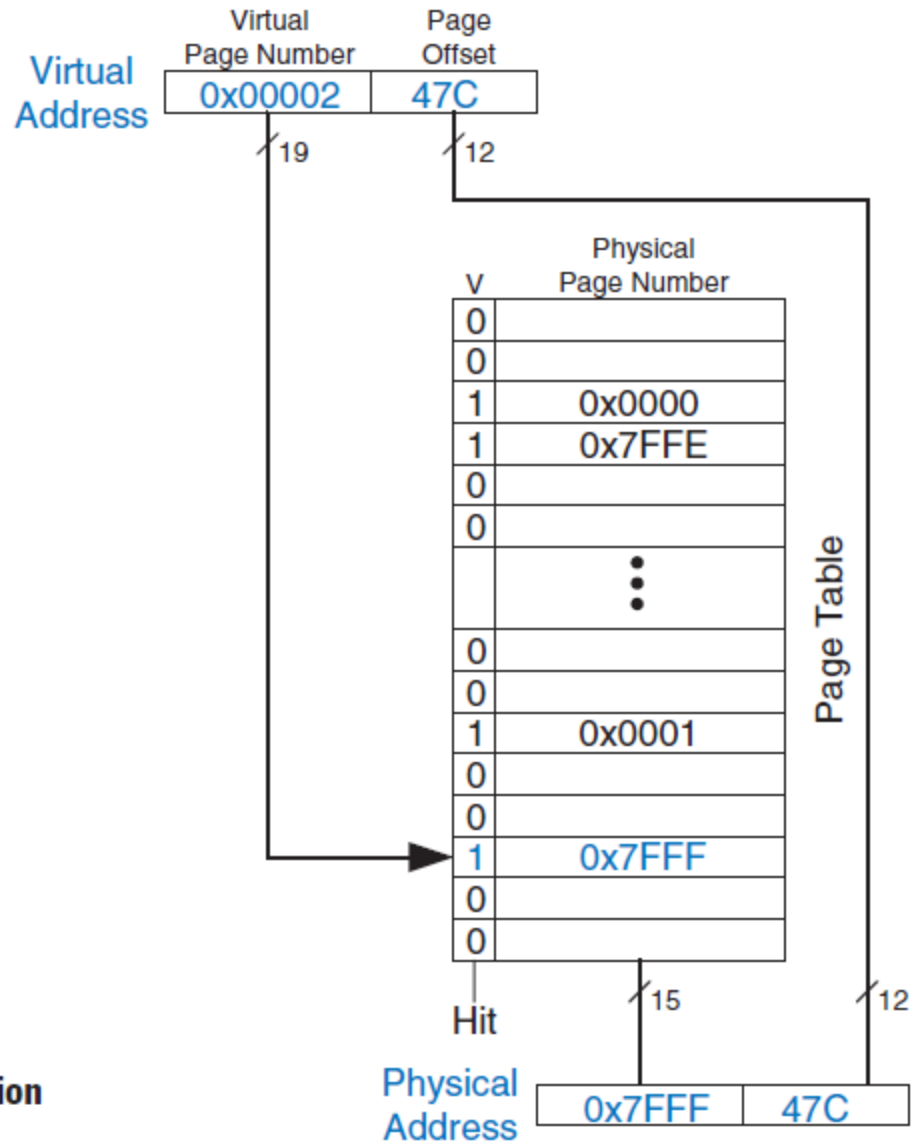

Figure 8.24 **Address translation using the page table**

- The page table can be stored anywhere in physical memory at the discretion of the OS.
- The processor typically uses a dedicated register, called the *page table register*, to store the base address of the page table in physical memory.
- To perform a load or store, the processor must first translate the virtual address to a physical address and then access the data at that physical address.
- The processor extracts the virtual page number from the virtual address and adds it to the page table register to find the physical address of the page table entry.
- The processor then reads this page table entry from physical memory to obtain the physical page number.
- If the entry is valid, it merges this physical page number with the page offset to create the physical address.
- Finally, it reads or writes data at this physical address.
- Because the page table is stored in physical memory, each load or store involves two physical memory accesses.

Translation Lookaside Buffer

- The processor can keep the last several page table entries in a small cache called a *translation lookaside buffer* (*TLB*).
- The processor “looks aside” to find the translation in the TLB before having to access the page table in physical memory.
- In real programs, the vast majority of accesses hit in the TLB, avoiding the time-consuming page table reads from physical memory.
- A TLB is organized as a fully associative cache and typically holds 16 to 512 entries.
- Each TLB entry holds a virtual page number and its corresponding physical page number.
- The TLB is accessed using the virtual page number.
- If the TLB hits, it returns the corresponding physical page number.
- Otherwise, the processor must read the page table in physical memory.
- The TLB is designed to be small enough that it can be accessed in less than one cycle.
- TLBs typically have a hit rate of greater than *99%*.
- The TLB decreases the number of memory accesses required for most load or store instructions from two to one.

Example 8.15 USING THE TLB TO PERFORM ADDRESS TRANSLATION

Q)Consider the virtual memory system of Figure 8.21. Use a two-entry TLB or explain why a page table access is necessary to translate virtual addresses 0x247C and 0x5FB0 to physical addresses. Suppose that the TLB currently holds valid translations of virtual pages 0x2 and 0x7FFFD.

Solution

- Figure 8.25 shows the two-entry TLB with the request for virtual address 0x247C.
- The TLB receives the virtual page number of the incoming address, 0x2, and compares it to the virtual page number of each entry.
- Entry 0 matches and is valid, so the request hits.
- The translated physical address is the physical page number of the matching entry, 0x7FFF, concatenated with the page offset of the virtual address.
- As always, the page offset requires no translation.
- The request for virtual address 0x5FB0 misses in the TLB.
- So, the request is forwarded to the page table for translation.

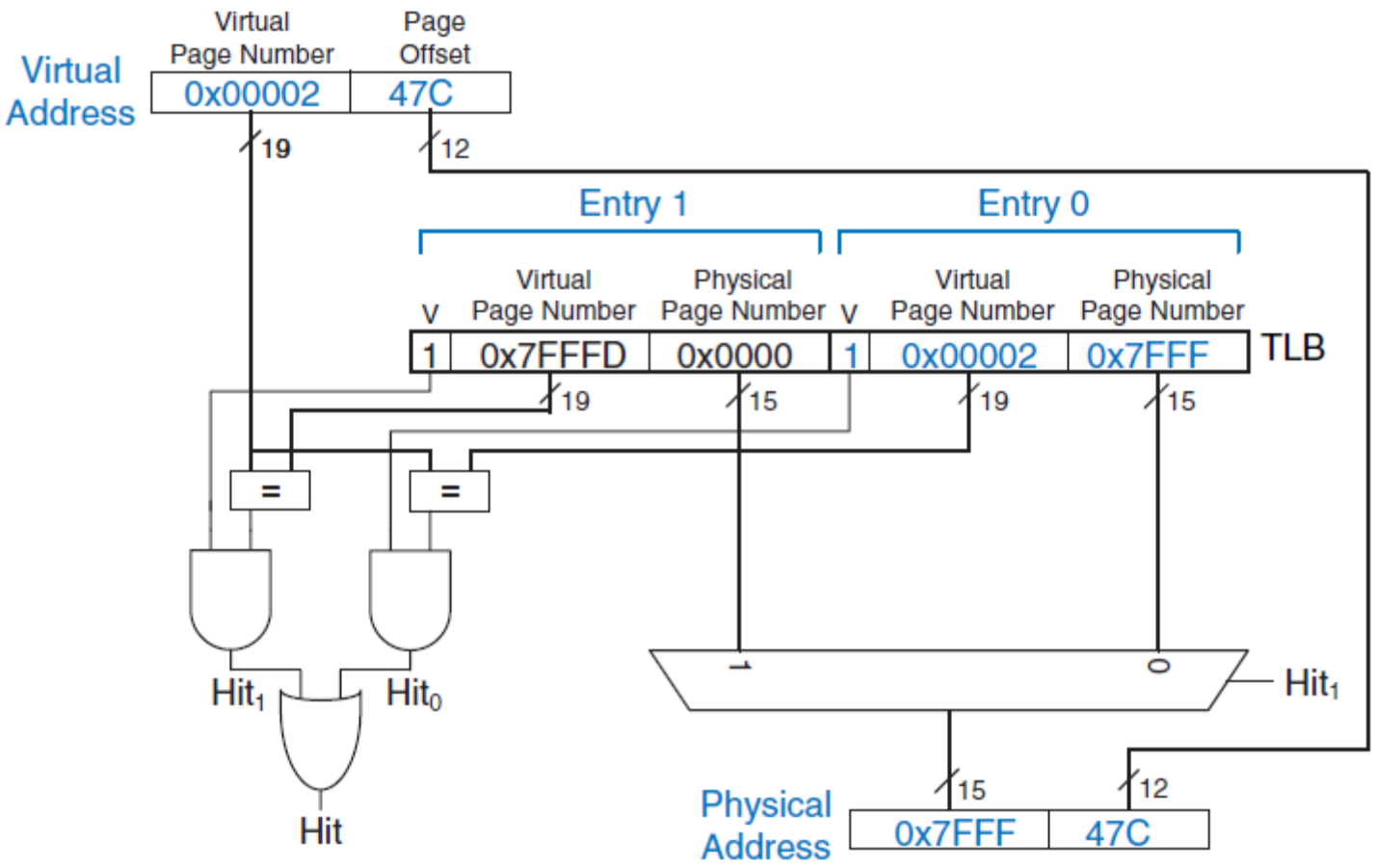

Figure 8.25 **Address translation using a two-entry TLB**

Memory Protection

- Virtual memory provide a fast, inexpensive, large memory.
- An equally important reason to use virtual memory is to provide protection between concurrently running programs.
- Modern computers typically run several programs or *processes* at the same time.
- All of the programs are simultaneously present in physical memory.
- In a well-designed computer system, the programs should be protected from each other so that no program can crash or hijack another program.
- Specifically, no program should be able to access another program's memory without permission.
- This is called *memory protection*.

- Virtual memory systems provide memory protection by giving each program its own *virtual address space*.
- Each program can use as much memory as it wants in that virtual address space, but only a portion of the virtual address space is in physical memory at any given time.
- Each program can use its entire virtual address space without having to worry about where other programs are physically located.
- However, a program can access only those physical pages that are mapped in its page table.
- In this way, a program cannot accidentally or maliciously access another program's physical pages because they are not mapped in its page table.
- In some cases, multiple programs access common instructions or data.
- The OS adds control bits to each page table entry to determine which programs, if any, can write to the shared physical pages.